

AI-Based Movie Recommendation System

Complete Project Files (Source Code & Data)

SUBMITTED BY

Rahul Singh

COURSE

Artificial Intelligence and Machine Learning

DATE

September 28, 2026

Contents

#	File	Description
1	main.py	Command-line interface
2	recommender.py	Recommendation engine (content-based, collaborative, hybrid)
3	generate_ratings.py	Generates the synthetic sample ratings
4	movies.csv	Movie catalog: 50 movies with genres
5	ratings.csv	Sample ratings: 12 users, 357 ratings
6	README.md	Project documentation
7	requirements.txt	Dependencies (standard library only)

1. main.py

Command-line interface

```
"""Movie recommender CLI.

Interactive:  python main.py
One-shot:    python main.py --interests Sci-Fi,Thriller --rate "Inception=5" "Toy Story=2" --top 5
Evaluate:    python main.py --evaluate
"""
import argparse
import sys

from recommender import Recommender, load_movies, load_ratings


def find_movie(movies, query: str):
    q = query.strip().lower()
    exact = [m for m in movies.values() if m.title.lower() == q]
    if exact:
        return exact[0]
    partial = [m for m in movies.values() if q in m.title.lower()]
    return partial[0] if len(partial) == 1 else None


def parse_interests(text: str, rec: Recommender) -> list[str]:
    lookup = {g.lower(): g for g in rec.genres}
    picked = []
    for part in text.split(","):
        g = lookup.get(part.strip().lower())
        if g:
            picked.append(g)
        elif part.strip():
            print(f" (ignoring unknown genre '{part.strip()}')")
    return picked


def show(results):
    if not results:
        print("No recommendations found.")
        return
    print("\nTop picks for you:\n")
    for i, (m, score, why) in enumerate(results, 1):
        print(f"{i}. {m.title} ({m.year}) - match {score}%")
        print(f"   {' '.join(m.genres)}")
        print(f"   Why: {why}\n")


def interactive(rec: Recommender):
    print("=== Movie Recommender ===")
    print("Genres:", ", ".join(rec.genres))
    interests = parse_interests(input("\nWhat genres are you into? (comma-separated): "), rec)

    ratings = {}
    print("\nRate movies you've seen (1-5), or press Enter to skip. Type 'done' to finish.")
    sample = sorted(rec.movies.values(), key=lambda m: -sum(m.movie_id in u for u in rec.ratings.values()))[:12]
    for m in sample:
        ans = input(f" {m.title} ({m.year}) [1-5/Enter]: ").strip().lower()
        if ans == "done":
            break
        if ans in {"1", "2", "3", "4", "5"}:
            ratings[m.movie_id] = float(ans)

    show(rec.recommend(interests, ratings, top_n=5))


def main():
    ap = argparse.ArgumentParser(description="Simple AI movie recommender")
    ap.add_argument("--interests", default="", help="comma-separated genres, e.g. Sci-Fi,Thriller")
    ap.add_argument("--rate", nargs="*", default=[], metavar="TITLE=RATING", help="e.g. 'Inception=5'")
    ap.add_argument("--top", type=int, default=5)
    ap.add_argument("--alpha", type=float, default=0.5, help="weight of content-based score (0-1)")
    ap.add_argument("--genre", help="only recommend this genre")
    ap.add_argument("--evaluate", action="store_true", help="report offline accuracy (RMSE)")
    args = ap.parse_args()

    rec = Recommender(load_movies(), load_ratings())

    if args.evaluate:
        model, base = rec.evaluate()
```

```

        print(f"RMSE collaborative model: {model:.3f}    global-mean baseline: {base:.3f}")
        return

if not args.interests and not args.rate:
    return interactive(rec)

ratings = {}
for item in args.rate:
    title, _, val = item.rpartition("=")
    movie = find_movie(rec.movies, title)
    if not movie or not val.replace(".", "", 1).isdigit():
        print(f"Could not use '{item}' (use TITLE=1-5 with a known title)", file=sys.stderr)
        continue
    ratings[movie.movie_id] = float(val)

show(rec.recommend(parse_interests(args.interests, rec), ratings,
                    top_n=args.top, alpha=args.alpha, genre_filter=args.genre))

if __name__ == "__main__":
    main()

```

2. recommender.py

Recommendation engine (content-based, collaborative, hybrid)

"""A simple hybrid movie recommender (pure Python, no dependencies).

Two signals are combined:

1. Content-based - movies are vectors of IDF-weighted genres. A user profile is built from the genres they like (stated interests + rated movies) and candidates are ranked by cosine similarity to that profile.
2. Collaborative - user-based k-NN: find existing users whose (mean-centred) ratings correlate with yours and predict what they'd say about unseen movies.

```
final_score = alpha * content + (1 - alpha) * collaborative
"""
```

```
from __future__ import annotations
```

```
import csv
import math
from dataclasses import dataclass
from pathlib import Path
```

```
DATA_DIR = Path(__file__).parent / "data"
```

```
NEUTRAL = 3.0 # a rating of 3/5 means "neutral"; above = like, below = dislike
```

```
@dataclass
```

```
class Movie:
    movie_id: int
    title: str
    year: int
    genres: list[str]
```

```
def load_movies(path: Path = DATA_DIR / "movies.csv") -> dict[int, Movie]:
    with open(path, newline="", encoding="utf-8") as f:
        return {
            int(r["movie_id"]): Movie(int(r["movie_id"]), r["title"], int(r["year"]), r["genres"].split("|"))
            for r in csv.DictReader(f)
        }
```

```
def load_ratings(path: Path = DATA_DIR / "ratings.csv") -> dict[int, dict[int, float]]:
    ratings: dict[int, dict[int, float]] = {}
    with open(path, newline="", encoding="utf-8") as f:
        for r in csv.DictReader(f):
            ratings.setdefault(int(r["user_id"]), {})[int(r["movie_id"])] = float(r["rating"])
    return ratings
```

```
def _cosine(a: dict, b: dict) -> float:
    dot = sum(v * b[k] for k, v in a.items() if k in b)
    na = math.sqrt(sum(v * v for v in a.values()))
    nb = math.sqrt(sum(v * v for v in b.values()))
    return dot / (na * nb) if na and nb else 0.0
```

```
def _mean(values) -> float:
    values = list(values)
    return sum(values) / len(values) if values else NEUTRAL
```

```
class Recommender:
    def __init__(self, movies: dict[int, Movie], ratings: dict[int, dict[int, float]]):
        self.movies = movies
        self.ratings = ratings
        self.genres = sorted({g for m in movies.values() for g in m.genres})
        self._build_movie_vectors()

    # ----- content-based -----
    def _build_movie_vectors(self) -> None:
        n = len(self.movies)
        df = {g: sum(1 for m in self.movies.values() if g in m.genres) for g in self.genres}
        self.idf = {g: math.log(n / df[g]) + 1 for g in self.genres} # rare genres count more
        self.vectors = {
            mid: {g: self.idf[g] for g in m.genres} for mid, m in self.movies.items()
        }

    def build_profile(self, interests: list[str], my_ratings: dict[int, float],
```

```

        interest_weight: float = 2.0) -> dict[str, float]:
    """Genre-preference vector: +weight for interests, +/- for liked/disliked movies."""
    profile: dict[str, float] = {}
    for g in interests:
        profile[g] = profile.get(g, 0) + interest_weight * self.idf.get(g, 1)
    for mid, rating in my_ratings.items():
        weight = rating - NEUTRAL
        for g, v in self.vectors[mid].items():
            profile[g] = profile.get(g, 0) + weight * v / len(self.vectors[mid])
    return profile

def content_scores(self, profile: dict[str, float]) -> dict[int, float]:
    return {mid: _cosine(profile, vec) for mid, vec in self.vectors.items()}

# ----- collaborative -----
def _similarity(self, mine: dict[int, float], other: dict[int, float]) -> float:
    common = mine.keys() & other.keys()
    if len(common) < 2:
        return 0.0
    m_mine, m_other = _mean(mine.values()), _mean(other.values())
    a = {k: mine[k] - m_mine for k in common}
    b = {k: other[k] - m_other for k in common}
    # shrink similarity when there are few co-rated movies
    return _cosine(a, b) * min(1.0, len(common) / 5)

def predict_rating(self, mine: dict[int, float], movie_id: int, k: int = 5,
                  exclude_user: int | None = None, damping: float = 1.0) -> float | None:
    """Predicted 1-5 rating from the k most similar users, or None if not enough data."""
    m_mine = _mean(mine.values())
    neighbours = []
    for uid, other in self.ratings.items():
        if uid == exclude_user or movie_id not in other:
            continue
        sim = self._similarity(mine, other)
        if sim > 0:
            neighbours.append((sim, other[movie_id] - _mean(other.values())))
    neighbours = sorted(neighbours, reverse=True)[:k]
    if not neighbours:
        return None
    # damping pulls weak-evidence predictions toward the user's average
    pred = m_mine + sum(s * d for s, d in neighbours) / (sum(s for s, _ in neighbours) + damping)
    return min(5.0, max(1.0, pred))

# ----- hybrid -----
def recommend(self, interests: list[str] | None = None, my_ratings: dict[int, float] | None = None,
             top_n: int = 5, alpha: float = 0.5, genre_filter: str | None = None):
    """Return [(Movie, score 0-100, explanation)] for movies the user hasn't rated."""
    interests = interests or []
    my_ratings = my_ratings or {}
    profile = self.build_profile(interests, my_ratings)
    content = self.content_scores(profile) if profile else {}
    use_cf = len(my_ratings) >= 3

    results = []
    for mid, movie in self.movies.items():
        if mid in my_ratings:
            continue
        if genre_filter and genre_filter not in movie.genres:
            continue
        c = content.get(mid, 0.0) # roughly 0..1
        cf_pred = self.predict_rating(my_ratings, mid) if use_cf else None
        if cf_pred is not None:
            cf = (cf_pred - 1) / 4 # map 1..5 -> 0..1
            score = alpha * max(c, 0) + (1 - alpha) * cf
        else:
            score = max(c, 0)
        results.append((movie, round(score * 100, 1), self._explain(movie, profile, cf_pred)))
    results.sort(key=lambda r: r[1], reverse=True)
    return results[:top_n]

def _explain(self, movie: Movie, profile: dict[str, float], cf_pred: float | None) -> str:
    liked = [g for g in movie.genres if profile.get(g, 0) > 0]
    parts = []
    if liked:
        parts.append("matches your taste in " + ", ".join(liked))
    if cf_pred is not None:
        parts.append(f"similar users would rate it ~{cf_pred:.1f}/5")
    return "; ".join(parts) or "popular pick"

# ----- evaluation -----

```

```

def evaluate(self, holdout: float = 0.2, seed: int = 0) -> tuple[float, float]:
    """RMSE of the collaborative predictor vs. a global-mean baseline on held-out ratings."""
    import random
    rng = random.Random(seed)
    global_mean = _mean(r for u in self.ratings.values() for r in u.values())
    se_model = se_base = n = 0
    for uid, user in self.ratings.items():
        items = list(user.items())
        rng.shuffle(items)
        cut = max(1, int(len(items) * holdout))
        test, train = items[:cut], dict(items[cut:])
        for mid, actual in test:
            pred = self.predict_rating(train, mid, exclude_user=uid)
            pred = global_mean if pred is None else pred
            se_model += (pred - actual) ** 2
            se_base += (global_mean - actual) ** 2
            n += 1
    return math.sqrt(se_model / n), math.sqrt(se_base / n)

```

3. generate_ratings.py

Generates the synthetic sample ratings

```
"""Generate a small synthetic ratings file (data/ratings.csv).
```

```
Each fake user has a favourite-genre profile; ratings are derived from how well  
a movie matches that profile, plus some noise. Fixed seed => reproducible.  
"""
```

```
import csv  
import random  
from pathlib import Path
```

```
DATA = Path(__file__).parent / "data"
```

```
PROFILES = {  
    "scifi_fan": {"Sci-Fi": 2, "Action": 1, "Thriller": 1, "Mystery": 1},  
    "romcom_fan": {"Romance": 2, "Comedy": 2, "Musical": 1, "Drama": 0.5},  
    "family_fan": {"Animation": 2, "Family": 2, "Adventure": 1, "Musical": 1, "Fantasy": 1},  
    "crime_buff": {"Crime": 2, "Thriller": 2, "Mystery": 1.5, "Drama": 1},  
    "horror_fan": {"Horror": 2.5, "Thriller": 1.5, "Mystery": 1},  
    "action_fan": {"Action": 2.5, "Adventure": 1.5, "Sci-Fi": 1, "Crime": 0.5},  
    "drama_lover": {"Drama": 2, "Biography": 1.5, "History": 1.5, "Romance": 1},  
    "fantasy_fan": {"Fantasy": 2.5, "Adventure": 2, "Family": 1, "Action": 0.5},  
    "cinephile": {"Drama": 1.5, "Mystery": 1.5, "Crime": 1, "Sci-Fi": 1, "Biography": 1},  
    "comedy_fan": {"Comedy": 2.5, "Romance": 1, "Family": 0.5},  
    "thrill_seeker": {"Thriller": 2, "Action": 1.5, "Horror": 1, "Crime": 1},  
    "musical_fan": {"Musical": 3, "Romance": 1, "Animation": 1, "Comedy": 0.5},  
}
```

```
def main():  
    rng = random.Random(42)  
    with open(DATA / "movies.csv", newline="", encoding="utf-8") as f:  
        movies = [(int(r["movie_id"]), r["genres"].split("|")) for r in csv.DictReader(f)]  
  
    rows = []  
    for user_id, (_, profile) in enumerate(PROFILES.items(), start=1):  
        seen = rng.sample(movies, k=rng.randint(22, 32)) # each user rated a subset  
        for movie_id, genres in seen:  
            affinity = sum(profile.get(g, 0) for g in genres) / len(genres)  
            rating = 2.2 + 1.6 * affinity + rng.gauss(0, 0.5)  
            rating = min(5, max(1, round(rating)))  
            rows.append((user_id, movie_id, rating))  
  
    with open(DATA / "ratings.csv", "w", newline="", encoding="utf-8") as f:  
        w = csv.writer(f)  
        w.writerow(["user_id", "movie_id", "rating"])  
        w.writerows(rows)  
    print(f"Wrote {len(rows)} ratings for {len(PROFILES)} users.")  
  
if __name__ == "__main__":  
    main()
```

4. movies.csv

Movie catalog: 50 movies with genres

movie_id,title,year,genres

```
1,The Matrix,1999,Action|Sci-Fi|Thriller
2,Inception,2010,Action|Sci-Fi|Thriller
3,Interstellar,2014,Adventure|Drama|Sci-Fi
4,The Dark Knight,2008,Action|Crime|Drama
5,Toy Story,1995,Animation|Comedy|Family
6,Finding Nemo,2003,Animation|Adventure|Family
7,The Godfather,1972,Crime|Drama
8,Pulp Fiction,1994,Crime|Thriller
9,Forrest Gump,1994,Comedy|Drama|Romance
10,Titanic,1997,Drama|Romance
11,The Notebook,2004,Drama|Romance
12,La La Land,2016,Comedy|Drama|Romance|Musical
13,Mad Max: Fury Road,2015,Action|Adventure|Sci-Fi
14,John Wick,2014,Action|Crime|Thriller
15,The Hangover,2009,Comedy
16,Superbad,2007,Comedy
17,Get Out,2017,Horror|Thriller|Mystery
18,The Conjuring,2013,Horror|Mystery|Thriller
19,A Quiet Place,2018,Horror|Sci-Fi|Thriller
20,Parasite,2019,Comedy|Drama|Thriller
21,Spirited Away,2001,Animation|Adventure|Family|Fantasy
22,The Lion King,1994,Animation|Adventure|Drama|Family|Musical
23,Frozen,2013,Animation|Adventure|Comedy|Family|Musical
24,The Lord of the Rings: The Fellowship of the Ring,2001,Adventure|Fantasy|Action
25,Harry Potter and the Sorcerer's Stone,2001,Adventure|Family|Fantasy
26,Gladiator,2000,Action|Adventure|Drama
27,Braveheart,1995,Action|Biography|Drama|History
28,Schindler's List,1993,Biography|Drama|History
29,The Shawshank Redemption,1994,Drama|Crime
30,Fight Club,1999,Drama|Thriller
31,Se7en,1995,Crime|Mystery|Thriller
32,The Silence of the Lambs,1991,Crime|Drama|Thriller|Horror
33,Avengers: Endgame,2019,Action|Adventure|Sci-Fi
34,Iron Man,2008,Action|Adventure|Sci-Fi
35,Blade Runner 2049,2017,Drama|Mystery|Sci-Fi
36,Arrival,2016,Drama|Mystery|Sci-Fi
37,Knives Out,2019,Comedy|Crime|Mystery
38,Gone Girl,2014,Drama|Mystery|Thriller
39,Coco,2017,Animation|Adventure|Family|Fantasy|Musical
40,Up,2009,Animation|Adventure|Comedy|Family
41,Whiplash,2014,Drama|Musical
42,The Social Network,2010,Biography|Drama
43,Dune,2021,Action|Adventure|Sci-Fi|Drama
44,Crazy Rich Asians,2018,Comedy|Drama|Romance
45,Bridesmaids,2011,Comedy|Romance
46,Jurassic Park,1993,Action|Adventure|Sci-Fi|Thriller
47,Back to the Future,1985,Adventure|Comedy|Sci-Fi
48,The Prestige,2006,Drama|Mystery|Sci-Fi|Thriller
49,Mamma Mia!,2008,Comedy|Musical|Romance
50,Casablanca,1942,Drama|Romance|History
```

5. ratings.csv

Sample ratings: 12 users, 357 ratings

user_id,movie_id,rating

1,8,3
1,2,4
1,48,5
1,18,4
1,16,2
1,15,2
1,9,3
1,7,3
1,35,3
1,6,2
1,38,3
1,28,2
1,3,4
1,49,2
1,41,1
1,14,3
1,45,1
1,33,3
1,37,2
1,47,3
1,43,4
1,23,2
1,21,2
1,29,2
1,31,3
1,42,2
1,50,2
1,34,4
1,19,3
1,44,3
1,1,4
1,36,4
2,42,3
2,5,4
2,39,2
2,41,3
2,11,3
2,35,2
2,16,5
2,46,2
2,30,3
2,25,2
2,18,2
2,36,3
2,15,5
2,21,3
2,4,3
2,38,2
2,3,3
2,37,3
2,26,3
2,9,5
2,34,3
2,7,2
2,19,3
2,23,3
2,43,2
2,29,2
2,33,2
2,44,5
2,13,3
2,24,3
2,45,4
2,49,5
3,33,4
3,49,3
3,12,3
3,50,2
3,7,2
3,41,3
3,20,2
3,45,1
3,47,3
3,39,5
3,13,2

3,10,2
3,24,4
3,11,2
3,35,2
3,34,3
3,1,2
3,21,4
3,32,2
3,36,4
3,4,2
3,48,2
3,27,2
3,26,3
3,43,3
3,8,3
4,36,3
4,15,2
4,38,4
4,49,3
4,1,4
4,5,2
4,41,3
4,4,4
4,47,2
4,45,1
4,3,3
4,22,3
4,44,2
4,33,2
4,16,2
4,18,5
4,32,4
4,14,4
4,9,3
4,24,1
4,30,5
4,29,5
4,19,3
4,28,3
4,50,3
4,8,5
4,26,3
4,37,5
4,7,4
4,43,3
4,21,2
5,25,2
5,17,5
5,30,3
5,19,5
5,28,2
5,45,2
5,36,3
5,43,3
5,32,4
5,10,1
5,13,3
5,47,2
5,14,3
5,4,2
5,35,3
5,37,2
5,21,2
5,44,2
5,33,3
5,39,2
5,16,2
5,49,2
6,20,2
6,30,2
6,21,3
6,5,2
6,1,4
6,49,3
6,40,2
6,37,3
6,7,3
6,47,4
6,35,3
6,14,3

6, 33, 4
6, 17, 2
6, 9, 2
6, 23, 3
6, 41, 2
6, 16, 2
6, 24, 4
6, 10, 3
6, 6, 3
6, 15, 3
6, 27, 4
6, 18, 2
6, 44, 3
6, 31, 2
6, 50, 2
6, 48, 4
6, 43, 5
6, 46, 4
6, 28, 2
6, 25, 3
7, 10, 5
7, 28, 5
7, 9, 4
7, 3, 4
7, 20, 3
7, 24, 2
7, 47, 3
7, 23, 2
7, 14, 2
7, 16, 2
7, 7, 3
7, 43, 3
7, 36, 3
7, 27, 4
7, 50, 5
7, 41, 4
7, 11, 5
7, 12, 3
7, 37, 2
7, 1, 3
7, 6, 2
7, 45, 3
7, 34, 2
7, 26, 4
7, 42, 5
7, 22, 3
7, 29, 4
7, 8, 3
7, 48, 3
7, 30, 4
8, 33, 3
8, 8, 2
8, 25, 5
8, 37, 2
8, 13, 3
8, 17, 2
8, 3, 3
8, 28, 2
8, 1, 2
8, 34, 4
8, 35, 3
8, 46, 3
8, 24, 5
8, 43, 2
8, 5, 3
8, 22, 2
8, 21, 4
8, 49, 3
8, 20, 2
8, 45, 2
8, 10, 2
8, 40, 3
8, 14, 2
8, 11, 3
8, 39, 5
8, 23, 3
8, 30, 3
8, 18, 2
8, 36, 2
8, 7, 4

8, 47, 4
9, 50, 4
9, 5, 2
9, 30, 3
9, 27, 3
9, 41, 2
9, 37, 4
9, 13, 2
9, 25, 2
9, 32, 3
9, 26, 2
9, 16, 2
9, 10, 4
9, 1, 2
9, 7, 4
9, 28, 3
9, 15, 2
9, 12, 3
9, 48, 4
9, 4, 3
9, 18, 3
9, 8, 3
9, 36, 5
9, 42, 4
9, 35, 5
9, 49, 2
9, 47, 2
9, 22, 3
9, 17, 3
9, 31, 3
9, 20, 3
9, 11, 4
10, 45, 5
10, 2, 3
10, 37, 4
10, 25, 2
10, 31, 2
10, 1, 2
10, 23, 3
10, 20, 3
10, 47, 4
10, 27, 2
10, 35, 2
10, 40, 3
10, 15, 5
10, 32, 2
10, 38, 3
10, 18, 2
10, 28, 2
10, 48, 2
10, 49, 4
10, 13, 1
10, 11, 2
10, 22, 2
10, 29, 3
10, 26, 2
10, 46, 1
10, 24, 2
10, 6, 1
10, 36, 2
10, 5, 4
10, 43, 2
10, 39, 3
11, 8, 5
11, 37, 2
11, 14, 4
11, 30, 4
11, 45, 2
11, 17, 4
11, 24, 4
11, 11, 1
11, 39, 3
11, 42, 3
11, 50, 2
11, 43, 2
11, 20, 3
11, 7, 2
11, 2, 4
11, 38, 2
11, 25, 2

11, 26, 4
11, 13, 2
11, 3, 2
11, 19, 3
11, 23, 3
11, 27, 3
11, 21, 3
11, 40, 2
11, 4, 3
11, 29, 3
11, 10, 2
11, 22, 2
11, 35, 2
11, 34, 3
11, 47, 2
12, 12, 4
12, 32, 3
12, 14, 2
12, 23, 4
12, 17, 2
12, 22, 4
12, 18, 1
12, 39, 4
12, 44, 3
12, 36, 3
12, 1, 2
12, 34, 3
12, 13, 2
12, 6, 3
12, 16, 3
12, 27, 3
12, 49, 5
12, 41, 4
12, 31, 2
12, 21, 3
12, 47, 2
12, 33, 1
12, 15, 3
12, 26, 2
12, 40, 3
12, 3, 2
12, 10, 3

6. README.md

Project documentation

AI Movie Recommendation System

A simple hybrid recommender that suggests movies based on **user interests** (favourite genres) and **ratings**. Pure Python, no dependencies.

How it works

Signal	Idea
Content-based	Movies are IDF-weighted genre vectors. Your profile = stated interests + genres of movies you rated high (minus those you rated low). Candidates are ranked by cosine similarity.
Collaborative	User-based k-NN: find users whose mean-centred ratings correlate with yours and predict your rating for unseen movies (with damping for weak evidence).
Hybrid	``score = alpha * content + (1 - alpha) * collaborative`` (default alpha = 0.5). With no ratings, it falls back to content-only (cold start).

Project layout

```
```\nmain.py          CLI (interactive + one-shot + evaluate)\nrecommender.py   recommendation engine\ngenerate_ratings.py creates synthetic data/ratings.csv (12 users, seed 42)\ndata/movies.csv  50 movies with genres\ndata/ratings.csv sample user ratings\n```\n
```

#### ## Usage

```
```\nbash\npython main.py          # interactive\npython main.py --interests Sci-Fi,Thriller --rate "Inception=5" "Toy Story=2" --top 5\npython main.py --interests Comedy --genre Romance --alpha 0.7\npython main.py --evaluate # hold-out RMSE vs. baseline\n```\n
```

Extending it

- Swap in the real [MovieLens](<https://grouplens.org/datasets/movielens/>) CSVs (same column layout).
- Add richer features (directors, cast, plot embeddings) to the movie vectors.
- Replace k-NN with matrix factorisation (e.g. SVD) or add a web UI (Flask/Streamlit).

7. requirements.txt

Dependencies (standard library only)

```
# No third-party dependencies - standard library only (Python 3.9+)
```